PDS/SDP OS internals 25/06/2024

4 An oS161 system is given, running on a sys161 MIPS simulator with 4MB of RAM memory.
For each of the following addresses, say if they can be a logical user address, a logical kernel address, a physical address (explain/motivate answers):

- 0x80803005: it cannot be a user logical address because it is > 2G. it cannot be a physical address because it is > 4M, it could be a logical kernel address, but it is not compatible with the RAM (it should be < 0x80400000)
- 0x312010: it can be both a logical user and a physical address because it is < 2G and <4M, respectively. It cannot be a kernel logical address
- 0x532100: similar to the previous one, but it cannot be a physical address as it is > 4M

Given a logical user address 0x4010, convert it to the related physical address. It is known that as->as_pbase1, as->as_pbase2, as->as_vbase1, as->as_vbase2, as->as_npages1, as->as_npages2 have the following values: 0x100000, 0x200000, 0x3000, 0x6000, 2, 4.
OS161 user addresses can be split with 12 bits (3 hex digits) for the page offset, so the address is in page 4, offset/displacement 0x10. As segment 1 spans within pages 3 and 4, the address is in segment 1. The physical address is 0x100000 + (0x4010 – 0x3000) = 0c101010.

Physical memory in dumbvm is allocated by multiple of a page, despite being a contiguous allocation scheme, because

- allocating by multiple of a page reduces internal fragmentation
  NO. It could increase it, as standard contiguous allocation (without page alignment/padding, has no internal fragmentation.

- the MMU in MIPS has a TLB, so logical-to-physical translation needs pages
  YES. As the MMU uses the TLB, translation needs a correspondence page-frame.

- dumbvm implements a page table
  NO. There is no page table in dombvm

- kmalloc can just allocate by multiple of pages.
  NO. Any size can be asked to kmalloc.

5 Consider the implementation of locks and condition variables in OS161. For each of the following sentences, say YES/NO and provide a motivation:

Function cv_wait receives a lock as a parameter because

- it is needed as parameter in the inner call to wchan_sleep NO. Function wchan_sleep needs a slinlock.
- the calling thread has to be the owner of the lock YES. This is one requirement of cv_wait
- the lock has to be released and acquired again by cv_wait YES: this is exactly what cv_wait should do with the lock
- the lock has to be released and acquired again by wchan_sleep NO: wchan_sleep works on a spinlock.

The lock can be implemented

- by a binary semaphore, without any other element/requirement NO. Because the semaphore does not handle ownership.
- by a binary semaphore, plus an additional element/requirement YES. The additional requirement is on ownership.
- by a condition variable NO. The condition variable NEEDS an existing lock and it is not thought for mutual exclusion
- by a wait channel YES, combined with a spinlock and with a proper management of ownership.

The implementation of function lock_acquire can include a call to

- functions spinlock_data_get and spinlock_data_testandset NO. The two functions are at a much lower layer: they are used to implement spinlocks.
- function P on a semaphore YES, if the lock is implemented by a semaphore.
- function cv_wait NO. The condition variable NEEDS an existing lock and it is not thought for mutual exclusion
- function wchan_sleep YES, in the implementation based on wait channel and spinlock